

TOAE209/TOAH209 – Design and Analysis of Algorithms

Week 4: Practical Exercises

Foreign Trade University

Instructions

For each problem below there is a starter file `exercises/starter_code_problemNN.py` containing function signatures with `raise NotImplementedError` bodies, and a small `__main__` block with tests. Implement the missing functions so the tests pass. A reference solution is provided in `solutions/solution_problemNN.py` – try the problem yourself before looking!

All problems use only the Python 3.10+ standard library. Shared helper functions live in `starter_code.py` at the week root: `generate_intervals`, `generate_jobs`, `is_feasible`, `interval_depth`, `total_length`, `schedule_lateness`.

The 22 problems are grouped into five themes:

- **Interval Scheduling** (Problems 1–6): the earliest-finish-time-first greedy algorithm, brute-force verification, and a counter-example showing why “shortest interval first” fails.
- **Interval Partitioning** (Problems 7–10): the earliest-start-time-first algorithm with a min-heap, and verification that it uses the minimum possible number of resources.
- **Minimizing Lateness** (Problems 11–15): the earliest-deadline-first algorithm, brute-force verification, a counter-example for “shortest job first”, and exchange-argument practice with inversions.
- **Optimal Caching** (Problems 16–19): the furthest-in-future (Belady) offline-optimal eviction policy, the online LRU heuristic, and brute-force verification of optimality.
- **Greedy on Other Structures** (Problems 20–22): coin change (canonical vs. non-canonical systems) and job sequencing with deadlines and profits.

1 Interval Scheduling

Problem 1 – Feasibility Checker for Interval Selections

Implement `is_feasible_scratch(selection)` **from scratch** (without using the helper of the same name in `starter_code.py`). A list of intervals `selection` (each a tuple `(start, finish)`) is *feasible* iff no two intervals overlap. Two intervals (s_1, f_1) and (s_2, f_2) overlap iff $s_1 < f_2$ and $s_2 < f_1$. Intervals that merely touch at an endpoint do **not** count as overlapping (intervals are half-open `[start, finish)`).

Example. `[(0,2),(2,4),(5,7)]` is feasible; `[(0,3),(2,5)]` is not (they overlap on `(2,3)`); `[(1,3),(3,6)]` is feasible (they only touch at 3).

Problem 2 – Interval Depth (Maximum Overlap)

Implement `interval_depth_scratch(intervals)` **from scratch**. The *depth* of a set of intervals is the maximum number of intervals that are simultaneously active at any point in time. Use a “sweep line”: create a `+1` event at each start time and a `-1` event at each finish time, sort all events by time (processing `-1` events *before* `+1` events at equal times, since intervals are half-open), and track the running sum – its maximum value is the depth.

Example. `[(0,3),(1,4),(2,5),(6,8),(7,9)]` has depth 3 (the first three intervals all overlap around $t = 2.5$).

Problem 3 – Earliest-Finish-Time-First Interval Scheduling

Implement `earliest_finish_time_first(intervals)`, the classic greedy algorithm for Interval Scheduling:

Sort requests by finish time.

Repeatedly:

 Select the request with the earliest finish time among those still compatible with everything selected so far.

 Discard all requests that overlap the selected request.

Return the selected intervals, in the order selected (increasing finish time). Use `is_feasible` from `starter_code.py` to sanity-check the result.

Problem 4 – Brute-Force Optimal Interval Scheduling

For small n , implement `brute_force_max_independent_set(intervals)` that returns the LARGEST feasible subset of `intervals`, by checking all 2^n subsets (via `itertools.combinations`, from largest size downward) and returning the first feasible one of maximum size. This is exponential and only intended for $n \leq 15$.

Problem 5 – Verifying Optimality of Earliest-Finish-Time-First

Implement `verify_eft_optimal(num_trials, n, seed)`: for each of `num_trials` random instances of n intervals (`generate_intervals(n, seed=seed + trial)`), compute the size of the greedy result (Problem 3) and the brute-force optimum (Problem 4). Return `True` iff these sizes are equal for every trial. This empirically confirms that earliest-finish-time-first is optimal.

Problem 6 – Counter-Example: Shortest-Interval-First Is Not Optimal

A tempting (but incorrect) greedy rule is “repeatedly pick the SHORTEST remaining compatible interval”. Implement `shortest_interval_first(intervals)` that does exactly this (sort by duration ascending, greedily keep feasible ones). Then implement `counter_example_instance()` returning a list of intervals on which this rule selects *strictly fewer* intervals than the optimum (Problem 4) – demonstrating that “shortest first” is not a valid greedy strategy for Interval Scheduling.

Hint. A single short interval can overlap (and thus block) two longer, mutually-compatible intervals.

2 Interval Partitioning

Problem 7 – Interval Partitioning (Earliest-Start-Time-First)

Implement `interval_partitioning(intervals)`: assign every interval to one of the smallest possible number of “resources” (e.g. classrooms) so that no two intervals on the same resource overlap.

Sort intervals by start time.

Maintain a min-heap of `(end_time, resource_id)` for resources in use.

For each interval `(s, f)` in order of start time:

 If the heap is non-empty and its minimum `end_time <= s`:

 pop that resource, assign `(s,f)` to it, push `(f, resource_id)`

 Else:

 allocate a new resource id, assign `(s,f)` to it, push `(f, new_id)`

Return a dict mapping each interval to its assigned resource id (0-indexed).

Problem 8 – Verifying Interval Partitioning Uses the Minimum Number of Resources

Implement `verify_partitioning_optimal(num_trials, n, seed)`: for each of `num_trials` random instances, compute the number of distinct resources used by `interval_partitioning` (Problem 7) and the depth via `interval_depth(starter_code.py)`. Return `True` iff these are equal for every trial – confirming that earliest-start-time-first always uses exactly `depth` resources, which is provably the minimum possible (since `depth` mutually-overlapping intervals each need a distinct resource).

Problem 9 – Can K Resources Suffice?

Implement `min_resources_needed(intervals)` that runs `interval_partitioning` (Problem 7) and returns the number of distinct resources used (WITHOUT calling `interval_depth` directly).

Then implement `can_schedule_with_k_resources(intervals, k)` returning `True` iff `min_resources_needed(intervals) <= k`.

Problem 10 – Stress-Testing Interval Partitioning on Random Instances

Implement `random_partitioning_is_valid(n, seed)`: generate n random intervals, run `interval_partitioning` (Problem 7), and check (a) no two intervals assigned to the same resource overlap, and (b) the resource ids used are exactly $\{0, 1, \dots, k-1\}$ for some k (contiguous, no gaps). Then implement `stress_test(num_trials, n, seed)` that returns `True` iff `random_partitioning_is_valid` holds for `num_trials` different random instances.

3 Minimizing Lateness

Problem 11 – Earliest-Deadline-First Scheduling

In the Minimizing Lateness problem, a single resource processes n jobs (d_i, t_i) (deadline, length), all available at time 0, without preemption. If job i finishes at time f_i , its *lateness* is $\max(0, f_i - d_i)$. The goal is to minimize the *maximum* lateness.

Implement `earliest_deadline_first(jobs)`: sort `jobs` (a list of `(deadline, length)` tuples) by deadline, ascending, and schedule them back-to-back in that order. Return `(order, max_lateness)` where `order` is the list of original indices in scheduling order, and `max_lateness` is computed via `schedule_lateness` (`starter_code.py`).

Example. For jobs (by index) with $(d_i, t_i) = (2, 1), (4, 2), (6, 3), (8, 4), (9, 3), (15, 2)$, EDF schedules them in index order 0, 1, 2, 3, 4, 5, with finish times 1, 3, 6, 10, 13, 15 and maximum lateness 4 (job 4 finishes at 13, deadline 9).

Problem 12 – Brute-Force Optimal Schedule for Minimizing Lateness

For small n , implement `brute_force_min_lateness(jobs)` that tries all $n!$ orderings (via `itertools.permutations`), computes the max lateness of each via `schedule_lateness`, and returns the minimum. Intended for $n \leq 8$.

Problem 13 – Verifying Optimality of Earliest-Deadline-First

Implement `verify_edf_optimal(num_trials, n, seed)`: for each of `num_trials` random instances of n jobs (`generate_jobs(n, seed=seed + trial)`), compute EDF's max lateness (Problem 11) and the brute-force optimum (Problem 12). Return `True` iff they are equal (within 10^{-9}) for every trial.

Problem 14 – Counter-Example: Shortest-Job-First Is Not Optimal for Lateness

Implement `shortest_job_first(jobs)`: sort jobs by *length*, ascending (ignoring deadlines), and schedule back-to-back, returning `(order, max_lateness)` as in Problem 11. Then implement `counter_example_instance()` returning a list of jobs on which `shortest_job_first` produces a strictly larger max lateness than `earliest_deadline_first` – e.g., a very short job with a very tight deadline, paired with a longer job with a much later deadline.

Problem 15 – Exchange Argument Practice: Inversions and Idle Time

The exchange-argument proof that EDF is optimal relies on: (1) an optimal schedule has no idle time, and (2) EDF produces a schedule with no *inversions* (a pair of jobs i scheduled before j with $d_i > d_j$).

Implement `count_inversions(jobs, order)`: count pairs of positions $a < b$ in `order` with `jobs[order[a]][0] > jobs[order[b]][0]`. Implement `has_idle_time(jobs, order)`: return `True` iff `order` is *not* a valid permutation of `range(len(jobs))` (a basic sanity check, since in this model idle time never naturally arises).

Verify: the EDF order has zero inversions; the fully-reversed order of $n = 6$ jobs has $\binom{6}{2} = 15$ inversions; swapping two adjacent jobs in the EDF order introduces exactly one inversion.

4 Optimal Caching

Problem 16 – Optimal Caching: Furthest-In-Future (Belady’s Algorithm)

In the Optimal Caching (offline) problem, a cache holds up to k items. Given a request sequence `seq`, implement `furthest_in_future(seq, k)` using the FURTHEST-IN-FUTURE (Belady’s MIN / LFD) eviction policy: on a miss with a full cache, evict the item whose *next* use in `seq` is furthest in the future (or never used again, counted as $+\infty$). Return the total number of cache misses.

Example. `furthest_in_future(["A","B","C","B","A"], k=2) == 4`.

Problem 17 – Online Caching: Least-Recently-Used (LRU)

Implement `least_recently_used(seq, k)`: on a miss with a full cache, evict the item whose most recent prior use is furthest in the *past*. Return the number of misses. LRU is *online* (no lookahead), unlike furthest-in-future.

Problem 18 – Comparing Furthest-In-Future and LRU

Implement `construct_bad_sequence_for_lru(k)` returning `(seq, k)` such that `least_recently_used(seq, k) > furthest_in_future(seq, k)` – i.e. LRU makes strictly more misses than the offline optimum on the same input.

Hint. A cyclic access pattern over $k + 1$ distinct items with a cache of size k , repeated several times, causes LRU to “thrash” (it always evicts the item that is about to be needed again), while furthest-in-future can do strictly better.

Problem 19 – Brute-Force Optimal Caching and Verifying FIF Optimality

For short sequences ($|seq| \leq 12$, $k \leq 3$), implement `brute_force_min_misses(seq, k)` via memoized recursion over `(position, frozenset(cache))`: at each position, either the requested item is already cached (no cost, recurse), or it is a miss (cost 1) – if the cache has room, add it; otherwise try evicting each possible item and take the best outcome. Return the overall minimum number of misses.

Then implement `verify_fif_optimal(seqs, k)` returning `True` iff `furthest_in_future` (Problem 16) matches `brute_force_min_misses` on every sequence in `seqs`.

5 Greedy on Other Structures

Problem 20 – Greedy Coin Change for a Canonical Coin System

Implement `greedy_change(amount, denominations)`: repeatedly take the largest denomination $\leq$ the remaining amount, as many times as possible, until the amount reaches 0. Return a dict `{denomination: count}`.

Then implement `brute_force_min_coins(amount, denominations)` via dynamic programming: `dp[0] = 0, dp[a] = 1 + min(dp[a-c] for c in denominations if c <= a)`.

For the canonical system `[1, 2, 5, 10, 20, 50, 100, 200, 500]`, greedy is **optimal**: verify `greedy_change` always uses the minimum number of coins for amounts `1, ..., 999`.

Problem 21 – A Non-Canonical Coin System Where Greedy Fails

Implement `find_greedy_failure(denominations, max_amount)`: search amounts `1, ..., max_amount` and return the smallest amount where `greedy_change` uses strictly more coins than `brute_force_min_coins`, or `None` if no such amount exists.

Then implement `non_canonical_system()` returning a list of denominations (including 1) for which a failure exists with `max_amount=100`.

Classic example. With denominations `{1, 3, 4}`: greedy makes 6 as `4 + 1 + 1` (3 coins), but `3 + 3` (2 coins) is optimal – the smallest failure is at amount 6.

Problem 22 – Job Sequencing with Deadlines and Profits

Each job i has a deadline d_i (positive integer) and profit p_i , takes exactly 1 unit of time, and earns its profit only if scheduled at or before its deadline (on a single machine with slots `1, 2, 3, ...`).

Implement `job_sequencing(jobs)`: sort jobs by profit *descending*; for each job, try to place it in the LATEST free slot t with $1 \leq t \leq d_i$ (placing as late as possible leaves earlier slots free for tighter-deadline jobs); skip the job if no such slot is free. Return `(schedule, total_profit)` where `schedule` maps slot $\rightarrow$ job index.

Then implement `is_feasible_schedule(jobs, schedule)`: `True` iff every scheduled job's slot is $\leq$ its deadline and all slots are distinct.

Example. Jobs $(d, p) = (2, 100), (1, 19), (2, 27), (1, 25), (3, 15)$. The greedy schedule achieves total profit $100 + 27 + 15 = 142$ using 3 of the 5 jobs.